DOC 07 · CAPATAZ AI

Arquitectura técnica y decisiones de ingeniería

Dashboard operativo • monorepo premium • LotOS UI ultra-plus

Plano técnico de ejecución para construir el dashboard con cimientos de acero: estructura del repo, reglas de ingeniería, estrategia de estado, observabilidad, despliegue continuo y evolución segura.

Documento: 07_Arquitectura_Tecnica_y_Decisiones_de_Ingeniería · Versión 1.0 · Marzo 2026

Objetivo: dejar un mapa técnico claro para desarrollar rápido sin fabricar deuda invisible.

Lectura rápida de decisiones madre

Tema	Decisión	Por qué gana
Repositorio	Monorepo con pnpm/Turborepo	Más velocidad para Victor + Ricardo, shared UI y tooling central
Frontend	Next.js App Router + Server Components + LotOS UI	Dashboard veloz, SSR donde conviene y DX limpia
Backend	Modular monolith dentro de app web + jobs aislables	Entrega rápida hoy, separación sana mañana
Estado	Server state con TanStack Query; UI state con Zustand	Sin inflar complejidad donde no hace falta
Validación	TypeScript estricto + Zod end to end	Menos fantasmas en runtime
Deploy	Vercel + Supabase + GitHub Actions	Preview instantáneo y release serio

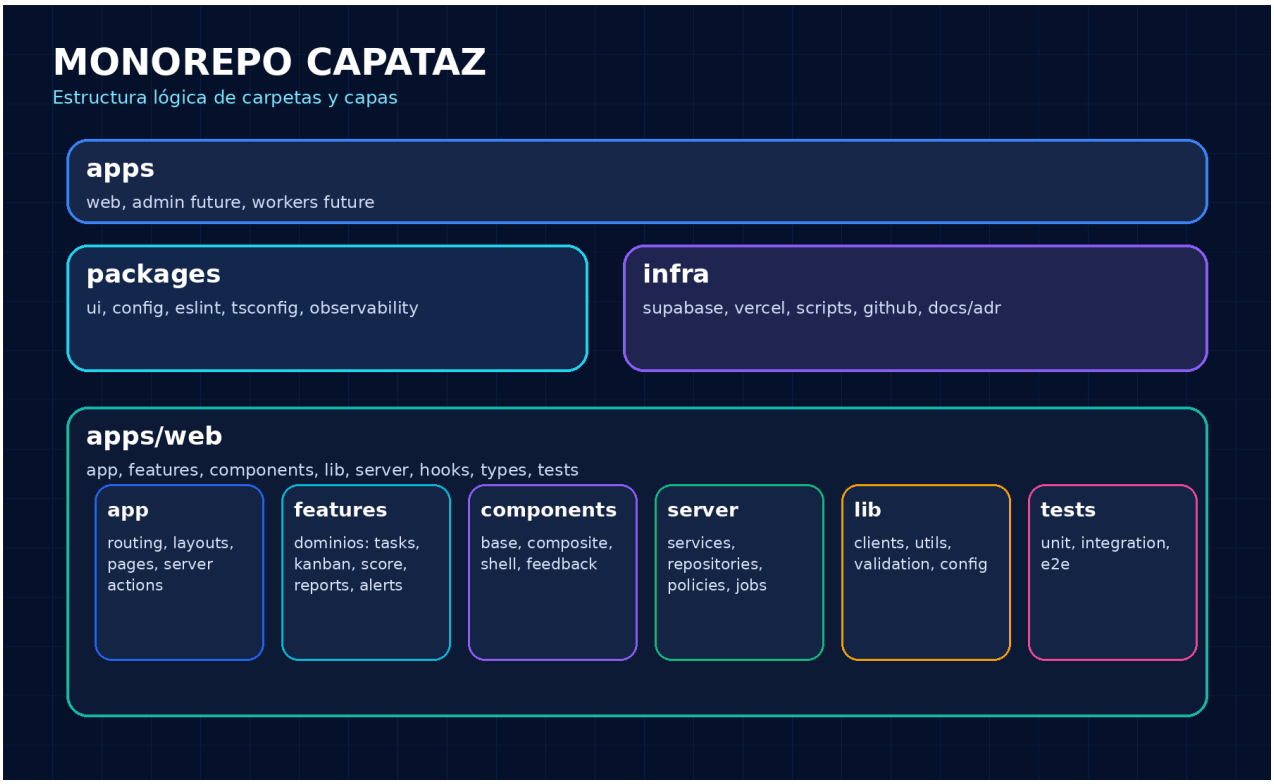
1. Principio técnico rector

Capataz no debe nacer como una ensalada de archivos brillantes. Debe nacer como un tablero operativo serio, con una arquitectura que permita entregar el MVP del dashboard rápido, pero sin hipotecar score, reportes, auditoría, tiempo real y canales futuros. La regla de oro es simple: monolito modular al inicio, fronteras claras desde el día uno, y evoluciones extraídas solo cuando la presión real del negocio lo exija.

- Diseñar para flujos operativos reales, no para una demo teatral.
- Separar dominio, UI, acceso a datos y observabilidad desde la base.
- Mantener shared tooling para que LotOS UI, validaciones y utilidades no se repliquen como gremlins.
- Preparar extracción futura de workers, alertas y conectores multicanal sin reescribir medio planeta.

2. Repositorio y estructura base

Decisión recomendada: monorepo. Para un equipo pequeño y rápido, un repo único reduce fricción, permite compartir paquetes de UI/configuración y facilita pipelines consistentes. Separar en varios repos desde el arranque sería un castillo con demasiados puentes levadizos.



- Gestor de paquetes: pnpm.
- Orquestación del workspace: Turborepo.
- Aplicación principal inicial: apps/web.
- Paquetes compartidos: packages/ui, packages/config, packages/eslint-config, packages/tsconfig, packages/observability.

- Infraestructura y documentación viva: infra/, docs/adr/, docs/runbooks/.

Árbol de referencia sugerido

```
/
├─ apps/
│  └─ web/
│     ├─ app/
│     ├─ components/
│     ├─ features/
│     ├─ hooks/
│     ├─ lib/
│     ├─ server/
│     ├─ styles/
│     ├─ types/
│     └─ tests/
├─ packages/
│  ├─ ui/
│  ├─ config/
│  ├─ eslint-config/
│  ├─ tsconfig/
│  └─ observability/
├─ infra/
│  ├─ supabase/
│  ├─ vercel/
│  └─ github/
└─ docs/
   ├─ adr/
   └─ runbooks/
```

3. Estrategia frontend

Next.js App Router como núcleo. Se aprovechan Server Components para cargar vistas con data inicial eficiente, Server Actions para mutaciones sencillas donde aporte claridad, y clientes React solo donde haya interacción intensa: Kanban, filtros dinámicos, scoreboards, edición inline y tiempo real.

- Capa shell en app/: layouts, rutas, segmentación y guards de navegación.
- Capa features/: cada dominio con UI, hooks, contratos y casos de uso visibles.
- LotOS UI como base visual. Los componentes de negocio se montan arriba, no al revés.
- Tailwind para velocidad, tokens centralizados para mantener identidad visual premium.
- Chart.js o ECharts para analítica. Para un dashboard de alto impacto, ECharts puede ser la opción premium si quieren mayor elasticidad visual.

4. Estrategia backend

Backend recomendado: modular monolith dentro de la app web, con fronteras explícitas por dominio. Nada de meter consultas SQL sueltas por toda la interfaz. Todo acceso a datos y reglas vive en server/.

- server/modules/tasks
- server/modules/kanban
- server/modules/score
- server/modules/reports
- server/modules/alerts
- server/modules/authz

Cada módulo debe contener servicios, políticas, repositorios, esquemas y transformadores. Cuando una capacidad crezca demasiado, se extrae a worker o microservicio con el mismo contrato. Así la migración futura deja de ser drama operístico.

5. Estrategia de estado y datos

- Server state: TanStack Query para vistas interactivas con cache, invalidación y refetch controlado.
- Mutaciones simples y formularios server-first: Server Actions donde el flujo sea directo y la experiencia lo permita.
- UI state efímero: Zustand para filtros, paneles, selección de tarjetas, preferencias locales y modos de vista.
- Form state: React Hook Form + Zod resolver.
- Tiempo real: Supabase Realtime solo para eventos valiosos, no para convertir cada pixel en una discoteca.

6. Estrategia de componentes UI

La UI debe tener jerarquía clara para que el sistema no se vuelva una caja de piezas mutantes.

- Base components: Button, Input, Select, Modal, Badge, Tabs, Table, Card, Avatar, Tooltip.
- Composite components: KanbanBoard, TaskCard, ScoreWidget, KPIHeader, ActivityFeed, FilterBar.
- Shell components: AppSidebar, AppTopbar, PageHeader, SectionPanel, CommandPalette.
- Pattern rules: un componente base nunca conoce reglas de negocio; un componente de negocio no reimplementa estilos base.

7. Convenciones de nombres, tipado y validación

- Componentes React: PascalCase. Ejemplo: TaskPriorityBadge.tsx.
- Hooks: camelCase con prefijo use. Ejemplo: useTaskFilters.ts.
- Servicios y repositorios: snake-free, explícitos. Ejemplo: taskScoreService.ts, taskRepository.ts.
- Zod schemas: *.schema.ts.
- Types y DTOs: *.types.ts o *.dto.ts.
- Routes y segmentos de app: kebab-case.
- Constantes globales: UPPER_SNAKE_CASE solo cuando realmente sean constantes.

TypeScript debe ir en modo estricto. Cero any casual. Si una zona necesita flexibilidad, se encapsula con unknown y refinamiento. Zod valida entradas de formularios, query params, payloads y contratos externos. El tipado no es maquillaje, es el detector de fugas del submarino.

8. Manejo de errores y resiliencia

- Errores de dominio: clases o códigos estables, traducibles a mensajes de negocio.
- Errores de infraestructura: se loguean con contexto técnico; al usuario se le muestra un mensaje útil y humano.
- Error boundaries en zonas de UI complejas.
- Fallbacks de datos para widgets del dashboard, evitando pantallas negras cuando falle una sola fuente.
- Retries solo donde tengan sentido. Nada de bucles histéricos.

9. Logging, auditoría y observabilidad

Recomendación: logging estructurado JSON con pino. Integrar Sentry para errores de frontend/backend y una capa de auditoría propia para eventos críticos del negocio.

- requestId y userId en cada operación relevante.
- Eventos auditables: crear tarea, asignar tarea, mover columna, cambiar prioridad, cerrar tarea, recalcular score, generar reporte.
- Dashboards de salud con Vercel Analytics, Supabase logs y Sentry.
- Runbooks en docs/runbooks para incidentes operativos.

Matriz de observabilidad

Señal	Herramienta	Retención inicial	Uso esperado
Errores UI/API	Sentry	30-90 días	Depurar fallos, stack traces, releases
Logs app	Pino + proveedor de logs	14-30 días	Correlación técnica y soporte
Auditoría negocio	Tabla propia en PostgreSQL	Largo plazo	Trazabilidad operativa y reportes
Performance web	Vercel Analytics	Variable	TTFB, navegación, Web Vitals

10. Ambientes, secrets y configuración

- Ambientes mínimos: local, preview, staging opcional, production.
- Variables por entorno en Vercel y Supabase, jamás hardcodeadas.
- Secretos sensibles solo en gestores de secretos del proveedor o GitHub Actions secrets.
- Separar claramente claves públicas de cliente y claves de servicio.
- Rotación documentada para tokens de WhatsApp/Meta, API keys de IA y service role keys.

11. CI/CD y despliegues

El pipeline debe ser automático y mezquino con la mala calidad.

- PR -> preview deployment en Vercel.
- Checks obligatorios: lint, typecheck, unit tests, build, validación de entorno.
- Merge a develop para integración continua interna.
- Main protegido para releases de producción.
- Migrations de base controladas con scripts y checklist de rollback.

12. Git, ramas y versionado

- main: producción.
- develop: integración del equipo.
- feature/*: trabajo de funcionalidad.
- fix/*: correcciones puntuales.
- release/*: estabilización y handoff previo a producción.
- hotfix/*: incendios reales en producción.

Versionado sugerido: Semantic Versioning. Mientras estén en MVP, pueden usar 0.x.y. Ejemplo: 0.4.0 para una nueva capacidad fuerte del dashboard; 0.4.2 para correcciones.

Tipo de cambio	Ejemplo	Versión
Nueva feature visible	Nuevo módulo de score por operador	minor
Fix sin ruptura	Arreglo de filtros y refresh	patch
Cambio incompatible	Reestructura de API interna	major

13. Decisiones de ingeniería iniciales (ADR shortlist)

- ADR-001: Monorepo con pnpm + Turborepo.
- ADR-002: Next.js App Router como aplicación principal.
- ADR-003: Supabase como base operacional inicial con PostgreSQL y Realtime.
- ADR-004: TypeScript estricto + Zod end to end.
- ADR-005: TanStack Query + Zustand como combinación de estado.
- ADR-006: Pino + Sentry + auditoría propia.
- ADR-007: Vercel preview deployments obligatorios para PR.

14. Recomendación final de stack ultra-plus

Si quieren “lo mejor de lo mejor” sin lanzarse a un laberinto, esta es la mezcla con mejor equilibrio entre velocidad, elegancia y capacidad futura: Next.js 15 + React 19, TypeScript estricto, Tailwind 4, LotOS UI como design language, pnpm + Turborepo, Supabase PostgreSQL + Realtime, Auth con Supabase, TanStack Query, Zustand, React Hook Form, Zod, Pino, Sentry, GitHub Actions y Vercel. Donde el negocio exija músculo extra, se agregan workers en Node o Bun para reportes, score batch y alertas multicanal.

Checklist para que el sprint arranque con botas puestas

Antes de programar	Definición visible
Workspace listo	pnpm workspace + turbo + scripts raíz
App web creada	Next.js con App Router y lint estricto
Paquete UI base	tokens, botón, card, input, modal, table
Módulo auth	roles base: admin, gerente, supervisor, operador
Esquemas iniciales	task, board, column, assignment, score_event, audit_log
Observabilidad	Sentry y logging estructurado desde el día 1
Pipeline	preview deploy por PR y main protegido